CHUYÊN ĐỀ: MẢNG CỘNG DỒN VÀ MẢNG HIỆU

Prefix Sum & Difference Array

Huỳnh Tấn Đức

Competitive Programming Training

June 17, 2026

Mục Lục

1. I. Mảng Cộng Dồn 1 Chiều (1D Prefix Sum)
2. II. Mảng Cộng Dồn 2 Chiều (2D Prefix Sum)
3. III. Mảng Hiệu (Difference Array)

I. Mảng cộng dồn 1 chiều (1D Prefix Sum)

1. Khái niệm - 1D Prefix Sum

Định nghĩa

Cho mảng A gồm N phần tử đánh số từ 1 đến N. Mảng cộng dồn S được định nghĩa là tổng các phần tử từ đầu đến vị trí i:

$$S[i] = A[1] + A[2] + \cdots + A[i] = \sum_{k=1}^{i} A[k]$$

Quy ước quan trọng (1-based indexing)

Gán $S[0] = 0$. Việc sử dụng chỉ số bắt đầu từ 1 giúp tránh việc xử lý chỉ số âm (Out-of-bounds) khi truy cập $S[i-1]$.

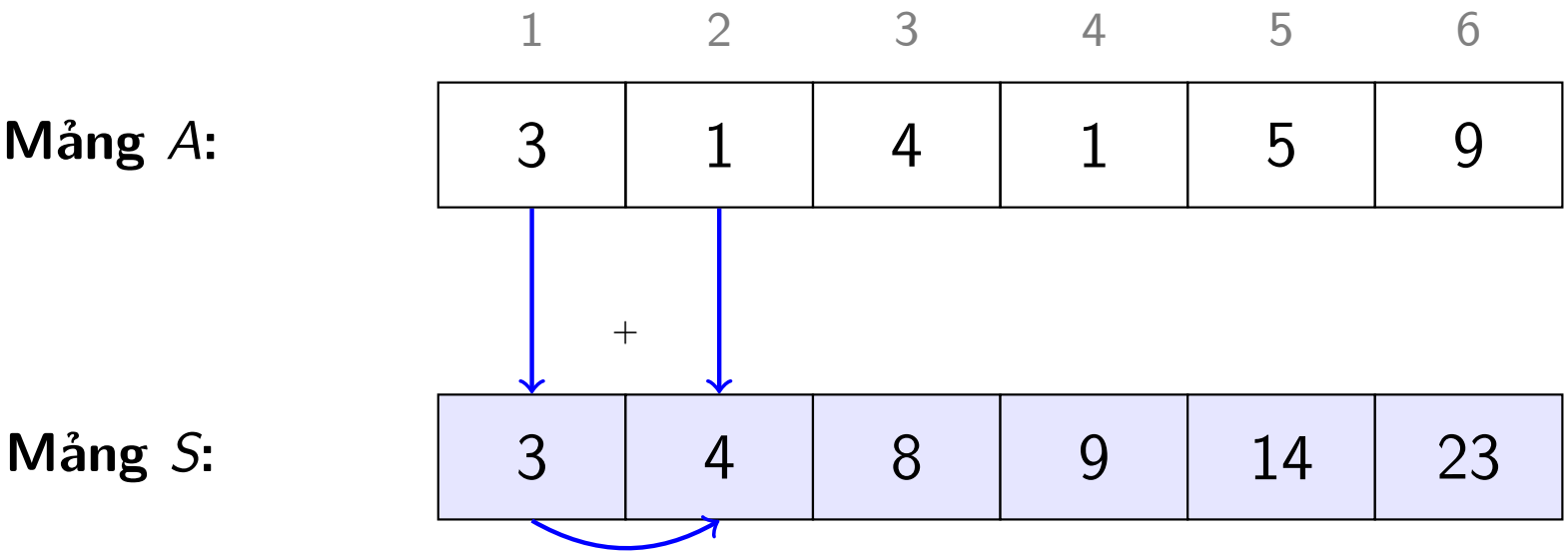
Ý tưởng trực quan

Thay vì tính lại tổng một đoạn bằng cách duyệt qua từng phần tử, ta lưu trước tổng cộng dồn để tái sử dụng ngay lập tức.

2. Ví dụ minh họa - 1D Prefix Sum

Chuyển đổi mảng A thành S

Giả sử mảng $A = [3, 1, 4, 1, 5, 9]$ (1-based index).



3. Thuật toán - 1D Prefix Sum

Các bước thực hiện

1. **Khởi tạo:** $S[0] = 0$.
2. **Xây dựng:** Duyệt i từ 1 đến N, áp dụng:
   $S[i] = S[i-1] + A[i]$
3. **Truy vấn:** Tính tổng đoạn $[L, R]$ bằng công thức:
   $\text{Sum}(L, R) = S[R] - S[L-1]$

Pseudocode

$S[0] \leftarrow 0$
FOR $i \leftarrow 1$ **TO** N :
  $S[i] \leftarrow S[i-1] + A[i]$
FUNCTION $Query(L, R)$:
  RETURN $S[R] - S[L-1]$

4. Độ phức tạp - 1D Prefix Sum

Thao tác	Time Complexity	Memory Complexity
Xây dựng mảng S	$\mathcal{O}(N)$	$\mathcal{O}(N)$
Truy vấn đoạn $[L, R]$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
Cách tính thủ công	$\mathcal{O}(R-L+1)$	$\mathcal{O}(1)$

- **Kết luận:** Prefix Sum đánh đổi bộ nhớ $\mathcal{O}(N)$ để tối ưu thời gian truy vấn xuống $\mathcal{O}(1)$, cực kỳ hiệu quả khi số lượng truy vấn Q lớn ($Q \times 1 \ll Q \times N$).

5. Code C++ - 1D Prefix Sum

Listing 1: Cài đặt 1D Prefix Sum

```cpp
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n, q; cin >> n >> q;
    vector<long long> A(n + 1), S(n + 1, 0);

    // Xay dung mang S
    for (int i = 1; i <= n; ++i) {
        cin >> A[i];
        S[i] = S[i-1] + A[i];
    }

    // Truy van doan [L, R]
    while (q--) {
        int L, R; cin >> L >> R;
        // O(1) Time
        cout << S[R] - S[L-1] << "\n";
    }
    return 0;
}
```

6. Ví dụ thực hành - 1D Prefix Sum

Đề bài

Cho mảng $A = [2, -1, 4, 3, 5]$. Trả lời 2 truy vấn tổng trong đoạn $[2, 4]$ và $[1, 5]$.

Giải thích từng bước

- Mảng A: `2, -1, 4, 3, 5`
- Mảng S: `0, 2, 1, 5, 8, 13` (1-based)

Truy vấn 1: Tổng đoạn $[2, 4]$

$$\text{Sum}(2, 4) = S[4] - S[1] = 8 - 2 = 6$$

(Thực tế: $-1 + 4 + 3 = 6 \rightarrow$ Đúng)

Truy vấn 2: Tổng đoạn $[1, 5]$

$$\text{Sum}(1, 5) = S[5] - S[0] = 13 - 0 = 13$$

7. Lỗi thường gặp - 1D Prefix Sum

Các sai lầm phổ biến

- **Lỗi tràn số (Integer Overflow):** Tổng các phần tử lớn vượt quá giới hạn của 'int' (32-bit).
- **Lỗi Out of bounds (0-based indexing):** Khi tính $S[R] - S[L-1]$ với $L = 0$ sẽ truy xuất $S[-1]$ gây lỗi bộ nhớ.

Cách khắc phục

- Sử dụng kiểu `long long` (64-bit) cho mảng S.
- Luôn dùng mảng đánh số từ 1 (**1-based indexing**) và khởi tạo cẩn thận $S[0] = 0$. Kích thước mảng cấp phát là $N + 1$.

II. Mảng cộng dồn 2 chiều (2D Prefix Sum

1. Khái niệm - 2D Prefix Sum

Định nghĩa

Cho ma trận A kích thước $N \times M$. Gọi $S[i][j]$ là tổng của hình chữ nhật con bắt đầu từ góc trái trên $(1, 1)$ đến góc phải dưới (i, j).

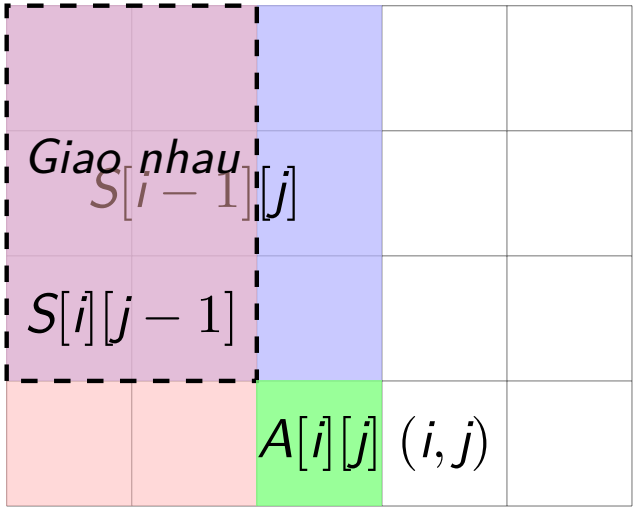
Ứng dụng

Dùng cho các bài toán truy vấn tổng hoặc tính trung bình hình chữ nhật con trên bảng số (ma trận, ảnh bitmap, grid 2D) nhanh chóng trong $\mathcal{O}(1)$.

2. Ví dụ minh họa & Ý tưởng trực quan

Nguyên lý Bao hàm - Loại trừ (Inclusion-Exclusion)

Được dùng để xây dựng và truy vấn để loại bỏ phần giao diện tích bị tính trùng.



$$S[i][j] = S[i-1][j] + S[i][j-1] - S[i-1][j-1] + A[i][j]$$

3. Thuật toán - 2D Prefix Sum

Công thức xây dựng

Lấy tổng phần trên cộng tổng phần trái, trừ đi phần giao nhau (bị cộng 2 lần) rồi cộng thêm ô hiện tại:

$$S[i][j] = S[i-1][j] + S[i][j-1] - S[i-1][j-1] + A[i][j]$$

Truy vấn tổng hình chữ nhật con

Để tính tổng hình chữ nhật có góc trái trên (x_1, y_1) và góc phải dưới (x_2, y_2):

$$\text{Sum} = S[x_2][y_2] - S[x_1-1][y_2] - S[x_2][y_1-1] + S[x_1-1][y_1-1]$$

(Áp dụng tương tự nguyên lý Bao hàm - Loại trừ cho việc lấy diện tích lớn trừ đi các phần thừa ngoài đoạn)

4. Độ phức tạp - 2D Prefix Sum

Thao tác	**Time Complexity**	**Memory Complexity**
Xây dựng ma trận S	$\mathcal{O}(N \times M)$	$\mathcal{O}(N \times M)$
Truy vấn hình chữ nhật	$\mathcal{O}(1)$	$\mathcal{O}(1)$
Tính thủ công 2 vòng 'for'	$\mathcal{O}(\Delta x \times \Delta y)$	$\mathcal{O}(1)$

- Việc xây dựng ban đầu tốn thời gian duyệt toàn bộ ma trận 1 lần.
- Truy vấn chỉ lấy đúng 4 phép toán cơ bản $\rightarrow$ cực kỳ tối ưu cho các bài toán xử lý ảnh, convolution matrix.

5. Code C++ - 2D Prefix Sum

Listing 2: Cài đặt 2D Prefix Sum

```cpp
// Gia su A[n+1][m+1] va S[n+1][m+1] da khoi tao bang 0
for (int i = 1; i <= n; ++i) {
    for (int j = 1; j <= m; ++j) {
        S[i][j] = S[i-1][j] + S[i][j-1] - S[i-1][j-1] + A[i][j];
    }
}

// Ham truy van hinh chu nhat tu (x1, y1) den (x2, y2)
long long query2D(int x1, int y1, int x2, int y2) {
    return S[x2][y2]
         - S[x1-1][y2]
         - S[x2][y1-1]
         + S[x1-1][y1-1];
}
```

6. Ví dụ thực hành - 2D Prefix Sum

Đề bài

Cho ma trận A 3×3. Tìm tổng HCN từ ô $(2, 2)$ đến $(3, 3)$.

$$A = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}$$

Giải thích

Mảng S xây dựng được:

$$S = \begin{pmatrix} 1 & 3 & 6 \\ 5 & 12 & 21 \\ 12 & 27 & 45 \end{pmatrix}$$

Tổng HCN $(2, 2) \to (3, 3)$:
Sum $= S[3][3] - S[1][3] - S[3][1] + S[1][1]$
Sum $= 45 - 6 - 12 + 1 = 28$
(Thử lại: $5 + 6 + 8 + 9 = 28 \to$ Chính xác)

7. Lỗi thường gặp - 2D Prefix Sum

Các sai lầm phổ biến

- **Quên cộng lại phần giao nhau:** Rất dễ viết thiếu $+S[i-1][j-1]$ trong cả xây dựng lẫn truy vấn.
- **Nhầm lẫn tọa độ** (x, y)**:** Trục toạ độ trong mảng 2D (dòng i, cột j) khác với toạ độ (x, y) Descartes học trong Toán học.

Cách khắc phục

- Thuộc kỹ nguyên tắc: **"Cộng to, trừ 2 mảnh nhỏ, cộng lại mảnh góc"**.
- Luôn làm việc theo hệ tọa độ Dòng-Cột (Row-Col), gọi biến là r_1, c_1, r_2, c_2 để tránh nhầm lẫn x, y.

III. Mảng hiệu (Difference Array)

1. Khái niệm - Difference Array

Định nghĩa Bài toán

Cho mảng A. Có Q thao tác, mỗi thao tác yêu cầu: **"Cộng thêm giá trị V vào tất cả phần tử thuộc đoạn $[L, R]$"**. Cuối cùng in ra mảng A.
Kỹ thuật Mảng hiệu là "ngược" của Prefix Sum. Mảng hiệu giỏi "Range Update" (Cập nhật đoạn).

Phân tích

Nếu dùng vòng lặp thông thường để duyệt từ $L \to R$ cộng V, độ phức tạp là $\mathcal{O}(Q \times N) \to$ Quá chậm, gây TLE.

2. Giải pháp & Ví dụ minh họa - Difference Array

Ý tưởng chính

Tạo mảng hiệu D sao cho: $D[i] = A[i] - A[i-1]$.
$\Rightarrow$ Mảng A chính là prefix sum của mảng D: $A[i] = \sum_{k=1}^{i} D[k]$.

Tác động của việc tăng đoạn

Giả sử đoạn $[L, R]$ đều tăng $+V$. Khoảng cách giữa $A[i]$ và $A[i-1]$ bên trong đoạn KHÔNG ĐỔI. Sự chênh lệch chỉ xảy ra ở 2 đầu biên:

- $A[L]$ đột ngột lớn hơn $A[L-1]$ một lượng $+V \rightarrow D[L]$ tăng V.
- $A[R+1]$ đứng im, trong khi $A[R]$ đã tăng $V \rightarrow D[R+1]$ giảm V.

3. Thuật toán - Difference Array

Các bước thực hiện

1. **Cập nhật (Range Update):** Thay vì tăng toàn bộ đoạn $[L, R]$, ta chỉ cập nhật 2 điểm trên mảng D:
   - $D[L] = D[L] + V$
   - $D[R+1] = D[R+1] - V$
2. **Khôi phục (Reconstruct):** Sau Q truy vấn, tính Prefix Sum của mảng D để khôi phục lại mảng A:

$$A[i] = A[i-1] + D[i]$$

Pseudocode

FUNCTION Update(L, R, V):
    D[L] $\leftarrow D[L] + V$
    $D[R+1] \leftarrow D[R+1] - V$
(Sau tất cả cập nhật)
FOR $i \leftarrow 1$ **TO** N :
    $D[i] \leftarrow D[i] + D[i-1]$ // Khôi phục A

4. Độ phức tạp - Difference Array

Thao tác	Time Complexity	Memory Complexity
Cập nhật đoạn $[L, R]$	$\mathcal{O}(1)$	$\mathcal{O}(1)$
Cập nhật thủ công	$\mathcal{O}(R - L + 1)$	$\mathcal{O}(1)$
Khôi phục mảng đích	$\mathcal{O}(N)$	$\mathcal{O}(N)$

- **Đánh giá:** Q thao tác cập nhật nay tốn $\mathcal{O}(Q \times 1) = \mathcal{O}(Q)$. Việc khôi phục ở bước cuối cùng tốn thêm $\mathcal{O}(N)$. Tổng thời gian $\mathcal{O}(N + Q)$ (tối ưu tuyệt đối so với $\mathcal{O}(Q \times N)$).

5. Code C++ - Difference Array

Listing 3: Cài đặt Difference Array

```cpp
#include <iostream>
#include <vector>
using namespace std;

int n, q;
vector<long long> A, D;

// Ham cap nhat doan [L, R] them V
void update(int L, int R, long long V) {
    D[L] += V;
    if (R + 1 <= n) {
        D[R+1] -= V;
    }
}

// Trong main(): Khoi phuc mang A
// D[i] duoc cap nhat ỹlu ếk D[i] = D[i] + D[i-1]
for (int i = 1; i <= n; ++i) {
    D[i] += D[i-1];
    A[i] = D[i]; // A ban dau = 0, gio nhan gia tri tu D
}
```

6. Ví dụ thực hành - Difference Array

Đề bài

Khởi tạo mảng A toàn 0 độ dài 5.
Thực hiện: 1) Update $[2, 4]$ thêm 3.
2) Update $[3, 5]$ thêm 2.

Giải thích từng bước

Mảng D ban đầu: `0, 0, 0, 0, 0, 0` (6 phần tử)

- Update $[2, 4], V = 3 \Rightarrow D[2] += 3,\ D[5] -= 3$.
  Mảng D: `0, 3, 0, 0, -3, 0`
- Update $[3, 5], V = 2 \Rightarrow D[3] += 2,\ D[6] -= 2$.
  Mảng D: `0, 3, 2, 0, -3, -2`

Khôi phục (cộng dồn) từ $i = 1 \ldots 5$:

- $A[1] = 0$
- $A[2] = 0 + 3 = 3$
- $A[3] = 3 + 2 = 5$
- $A[4] = 5 + 0 = 5$

7. Lỗi thường gặp - Difference Array

Các sai lầm phổ biến

- **Cập nhật sai mút R:** Thay vì $D[R+1] -= V$, nhiều người viết nhầm $D[R] -= V$. Điều này triệt tiêu mất giá trị ở $A[R]$.
- **Truy xuất ngoài mảng:** Khi $R = N$, việc gọi $D[R+1]$ tức $D[N+1]$ sẽ bị tràn mảng (SegFault) nếu không cấp phát đủ lớn.

Cách khắc phục

- Luôn nhớ: **Tăng bắt đầu ở L, kết thúc hiệu lực ở $R+1$.**
- Luôn khởi tạo kích thước mảng là $N+2$ thay vì N để an toàn khi gọi $R+1$ lúc $R = N$.

CẢM ƠN CÁC BẠN ĐÃ THEO DÕI!

Q&A